

Análisis Estadístico Descriptivo e Inferencial

LABORATORIO 1

Autor: Sebastian Chavez Orellana

Dataset “Seeds” - Introducción

- El dataset “**Seeds**” es un conjunto de datos clásico utilizado en problemas de clasificación.
- Fue recolectado por investigadores para estudiar las características físicas de **tres variedades de semillas**.
- Contiene **7 atributos** medidos en cada semilla, además de una columna que indica la **variedad**.
- Los atributos incluyen medidas como:
 - Área
 - Perímetro
 - Compacidad
 - Longitud del núcleo
 - Ancho del núcleo
 - Coeficiente de asimetría (AC)
 - Longitud del surco (Length Groove)

Aplicaciones del Dataset

- El dataset es comúnmente utilizado para practicar y evaluar **técnicas de clasificación** en Machine Learning, como **k-NN**, **Árboles de Decisión** y **Regresión Logística**.
- Se utiliza también para ejercicios de **análisis exploratorio** y **visualización de datos**.

Carga y Visualización de Datos del Dataset “Seeds”

```

1 # Definir los nombres de las columnas del dataset
2 columns = c("area", "perimeter", "compactness", "length", "width", "AC", "lengthGroove", "class")
3
4 # Especificar la URL del archivo CSV que contiene el dataset "Seeds"
5 url = "https://www.dl.dropboxusercontent.com/s/wrexlo5im3g5ioi/seeds_dataset.csv"
6
7 # Leer los datos del CSV desde la URL y asignar los nombres de las columnas
8 seeds = read.csv(url, header = F, sep=";", col.names = columns)
9
10 # Mostrar las primeras filas del dataset para verificar la carga de datos
11 head(seeds)

```

	area	perimeter	compactness	length	width	AC	lengthGroove	class
1	15.26	14.84	0.8710	5.763	3.312	2.221	5.220	1
2	14.88	14.57	0.8811	5.554	3.333	1.018	4.956	1
3	14.29	14.09	0.9050	5.291	3.337	2.699	4.825	1
4	13.84	13.94	0.8955	5.324	3.379	2.259	4.805	1
5	16.14	14.99	0.9034	5.658	3.562	1.355	5.175	1
6	14.38	14.21	0.8951	5.386	3.312	2.462	4.956	1

Carga y Visualización de Datos del Dataset “Seeds”

```
1 tail(seeds)
```

	area	perimeter	compactness	length	width	AC	lengthGroove	class
205	12.37	13.47	0.8567	5.204	2.960	3.919	5.001	3
206	12.19	13.20	0.8783	5.137	2.981	3.631	4.870	3
207	11.23	12.88	0.8511	5.140	2.795	4.325	5.003	3
208	13.20	13.66	0.8883	5.236	3.232	8.315	5.056	3
209	11.84	13.21	0.8521	5.175	2.836	3.598	5.044	3
210	12.30	13.34	0.8684	5.243	2.974	5.637	5.063	3

Carga y Visualización de Datos del Dataset “Seeds”

```
1 str(seeds)
```

```
'data.frame':  210 obs. of  8 variables:
 $ area      : num  15.3 14.9 14.3 13.8 16.1 ...
 $ perimeter : num  14.8 14.6 14.1 13.9 15 ...
 $ compactness : num  0.871 0.881 0.905 0.895 0.903 ...
 $ length    : num  5.76 5.55 5.29 5.32 5.66 ...
 $ width     : num  3.31 3.33 3.34 3.38 3.56 ...
 $ AC        : num  2.22 1.02 2.7 2.26 1.35 ...
 $ lengthGroove: num  5.22 4.96 4.83 4.8 5.17 ...
 $ class     : int   1 1 1 1 1 1 1 1 1 ...
```

Transformación

```
1 seeds$class = factor(seeds$class, levels = c(1,2,3), labels = c("Kama", "Rosa", "Canadian"))
2 str(seeds)
```

```
'data.frame':  210 obs. of  8 variables:
 $ area      : num  15.3 14.9 14.3 13.8 16.1 ...
 $ perimeter  : num  14.8 14.6 14.1 13.9 15 ...
 $ compactness : num  0.871 0.881 0.905 0.895 0.903 ...
 $ length     : num  5.76 5.55 5.29 5.32 5.66 ...
 $ width      : num  3.31 3.33 3.34 3.38 3.56 ...
 $ AC         : num  2.22 1.02 2.7 2.26 1.35 ...
 $ lengthGroove: num  5.22 4.96 4.83 4.8 5.17 ...
 $ class      : Factor w/ 3 levels "Kama","Rosa",...: 1 1 1 1 1 1 1 1 1 1 ...
```

Sintaxis

```
1 factor(x = character(),           # Vector de entrada
2      levels,                     # Valores únicos de x (opcional)
3      labels = levels,             # Etiquetas para los niveles (opcional)
4      exclude = NA,               # Valores a excluir de los niveles
5      ordered = is.ordered(x),    # Si los niveles de entrada están ordenados o no
6      nmax = NA)                  # Máximo número de niveles
```

Resumen

```
1 summary(seeds)
```

area	perimeter	compactness	length
Min. :10.59	Min. :12.41	Min. :0.8081	Min. :4.899
1st Qu.:12.27	1st Qu.:13.45	1st Qu.:0.8569	1st Qu.:5.262
Median :14.36	Median :14.32	Median :0.8734	Median :5.524
Mean :14.85	Mean :14.56	Mean :0.8710	Mean :5.629
3rd Qu.:17.30	3rd Qu.:15.71	3rd Qu.:0.8878	3rd Qu.:5.980
Max. :21.18	Max. :17.25	Max. :0.9183	Max. :6.675

width	AC	lengthGroove	class
Min. :2.630	Min. :0.7651	Min. :4.519	Kama :70
1st Qu.:2.944	1st Qu.:2.5615	1st Qu.:5.045	Rosa :70
Median :3.237	Median :3.5990	Median :5.223	Canadian:70
Mean :3.259	Mean :3.7002	Mean :5.408	
3rd Qu.:3.562	3rd Qu.:4.7687	3rd Qu.:5.877	
Max. :4.033	Max. :8.4560	Max. :6.550	

1. **vars**: Número de variables en el data frame o conjunto de datos analizado.
2. **n**: Número de observaciones disponibles para cada variable. Para el caso 210.
3. **mean**: Media aritmética de cada variable.
4. **sd**: Desviación estándar de cada variable. Esta mide la dispersión de los datos alrededor de la media.
5. **median**: La mediana de cada variable, es el valor en el cual se encuentra el punto medio de un conjunto de datos ordenados.
6. **trimmed**: Medida recortada de cada variable. Es la media calculada después de eliminar un cierto porcentaje de los valores extremos en cada extremo del conjunto de datos.
7. **mad**: Desviación absoluta de la mediana de cada variable. Medida de dispersión robusta similar a la desviación estándar pero menos sensible a los valores atípicos.
8. **min**: Es el valor mínimo observado para cada variable.
9. **max**: Es el valor máximo observado para cada variable.
10. **range**: Es el rango de cada variable. Se calcula como la diferencia entre el valor máximo y el valor mínimo.
11. **skew**: Asimetría de cada variable. Indica la falta de simetría en la distribución de los datos.
12. **kurtosis**: Curtosis de cada variable, mide la forma de la distribución de los datos. Valores positivos indican una distribución más puntiaguda (más picos) que una distribución normal, mientras que valores negativos indican una distribución más aplanada.
13. **se**: Error estándar de la media para cada variable. Indica la precisión de la estimación de la media muestral.


```

1 # Seleccionar solo las columnas numéricas
2 numeric_seeds <- seeds[, sapply(seeds, is.numeric)]
3
4 # Calcular la correlación
5 correlation_matrix <- cor(numeric_seeds)
6 correlation_matrix

```

	area	perimeter	compactness	length	width
area	1.0000000	0.9943409	0.6082884	0.9499854	0.9707706
perimeter	0.9943409	1.0000000	0.5292436	0.9724223	0.9448294
compactness	0.6082884	0.5292436	1.0000000	0.3679151	0.7616345
length	0.9499854	0.9724223	0.3679151	1.0000000	0.8604149
width	0.9707706	0.9448294	0.7616345	0.8604149	1.0000000
AC	-0.2295723	-0.2173404	-0.3314709	-0.1715624	-0.2580365
lengthGroove	0.8636927	0.8907839	0.2268248	0.9328061	0.7491315

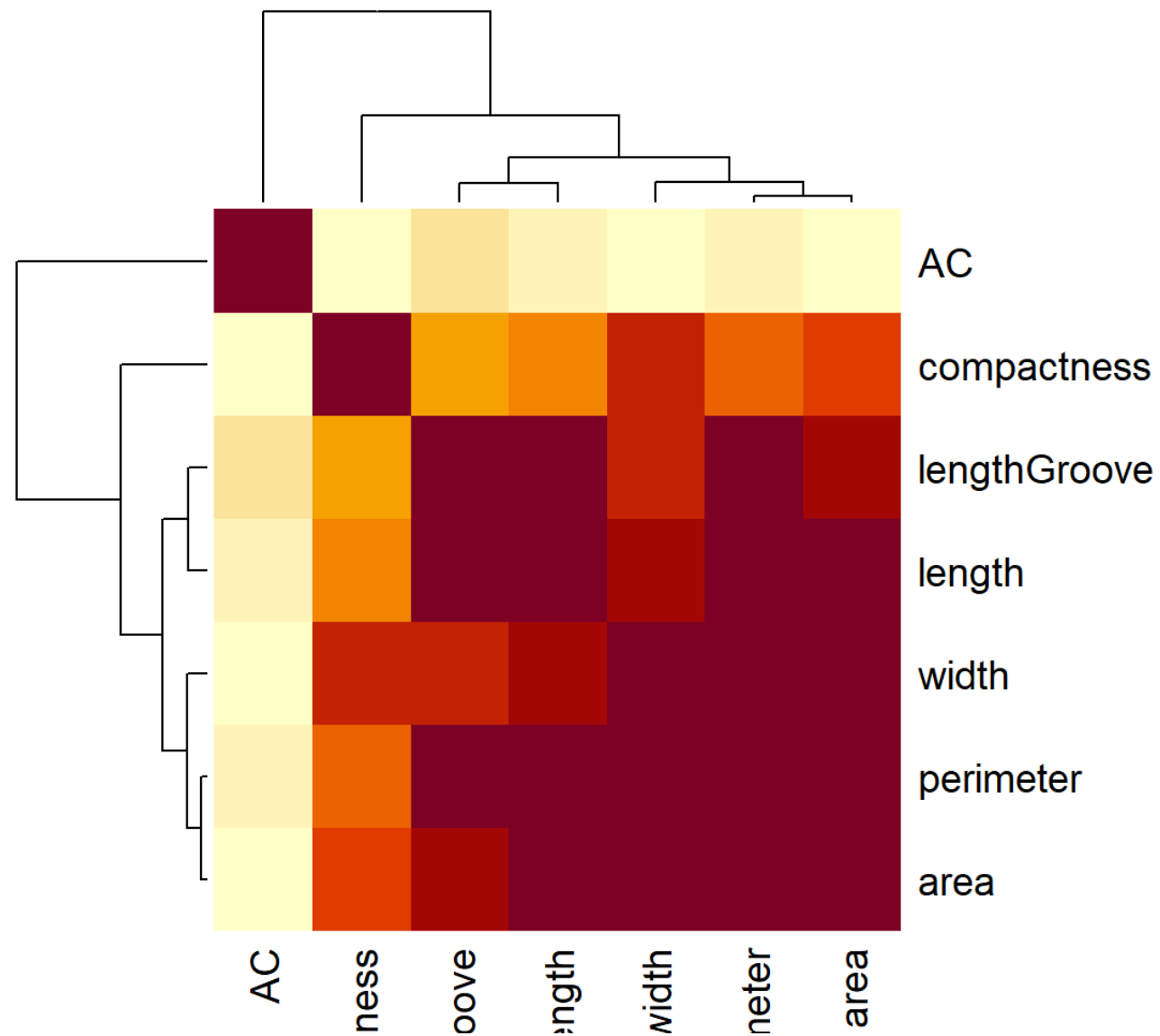
	AC	lengthGroove
area	-0.22957233	0.86369275
perimeter	-0.21734037	0.89078390
compactness	-0.33147087	0.22682482
length	-0.17156243	0.93280609
width	-0.25803655	0.74913147
AC	1.00000000	-0.01107902
lengthGroove	-0.01107902	1.00000000

La línea `sapply(seeds, is.numeric)` aplica una verificación para identificar qué columnas del data frame `seeds` son numéricas, devolviendo un vector de valores `TRUE` o `FALSE`. Luego, la notación de subconjuntos `seeds[, sapply(seeds, is.numeric)]` se emplea para seleccionar y mantener solo esas columnas numéricas de `seeds`, incluyendo todas sus filas.

```

1 heatmap(correlation_matrix,
2         symm = TRUE,
3         margins = c(3, 4))

```



Outliers

```
1 detectar_valores_atipicos_tukey <- function(data, columna) {  
2   # Calcular los cuartiles Q1 y Q3  
3   q1 <- quantile(data[[columna]], 0.25)  
4   q3 <- quantile(data[[columna]], 0.75)  
5   # Calcular el rango intercuartílico (IQR)  
6   iqr <- q3 - q1  
7   # Calcular los límites para detectar valores atípicos según el método de Tukey  
8   limite_inferior <- q1 - 1.5 * iqr  
9   limite_superior <- q3 + 1.5 * iqr  
10  # Identificar los valores atípicos  
11  ati_tukey <- data[[columna]] < limite_inferior | data[[columna]] > limite_superior  
12  # Obtener los índices de los valores atípicos  
13  indices_atipicos <- which(ati_tukey)  
14  # Retornar los índices de los valores atípicos  
15  return(indices_atipicos)  
16 }
```

Outliers

```

1 columnas <- c("area", "perimeter", "compactness", "length", "width", "AC", "lengthGroove")
2
3 for (col in columnas) {
4   indices_atipicos <- detectar_valores_atipicos_tukey(seeds, col)
5   cat("Columna:", col, "\n")
6   cat("Índices de valores atípicos:", indices_atipicos, "\n\n")
7 }

```

Columna: area

Índices de valores atípicos:

Columna: perimeter

Índices de valores atípicos:

Columna: compactness

Índices de valores atípicos: 158 159 175

Columna: length

Índices de valores atípicos:

Columna: width

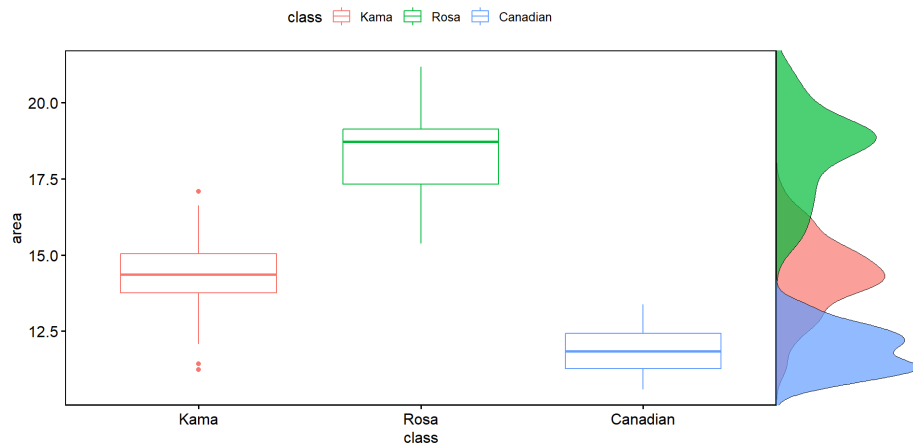
Índices de valores atípicos:

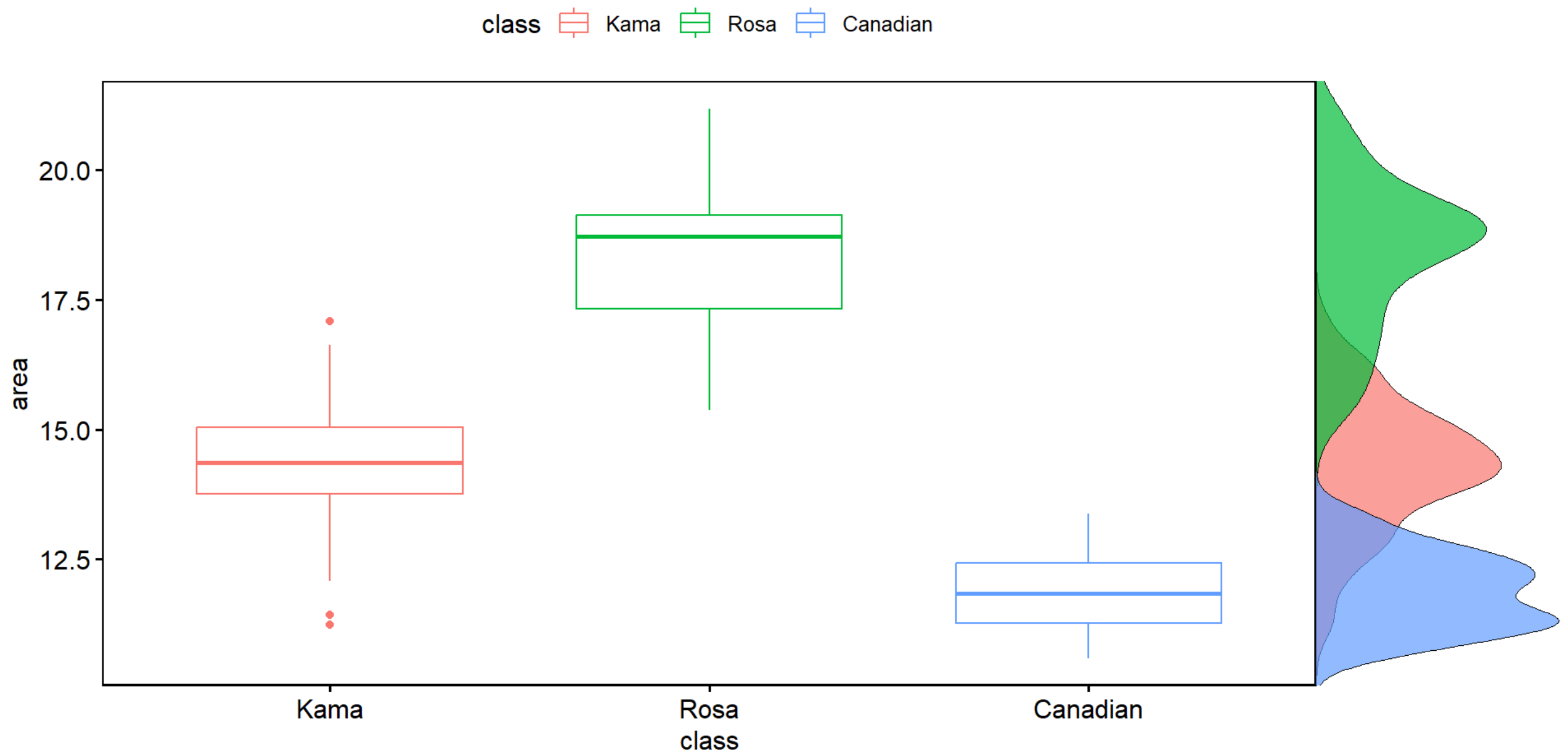
Columna: AC

Gráfico de cajas

`boxplot.width <- ggboxplot(data = seeds, x = "class", y = "width", color = "class") + border()`. Gráfico de cajas en R utilizando la librería `ggplot2`. `ggboxplot()`. Función que genera el gráfico, se designa `seeds` como la fuente de datos. La variable “class” se usa para formar los grupos en el eje x, mientras que “width” es la medida que se muestra en el eje y. Además, “class” coloreará los grupos de forma diferenciada y `+ border()` se añade para delinear cada elemento gráficamente.

```
1 boxplot.width <- ggboxplot(data = seeds, x = "class", y = "area", color = "class") + border()
2 ydens <- axis_canvas(boxplot.width, axis = "y", coord_flip = TRUE) +
3   geom_density(data = seeds, aes(x = get("area"), fill = get("class")), alpha = 0.7, linewidth = 1)
4   coord_flip()
5 boxplot.width <- insert_yaxis_grob(boxplot.width, ydens, grid::unit(.2, "null"), position = "right")
6 ggdraw(boxplot.width)
```





Prueba t de Student para Comparar Grupos

- Para comparar las **medias de dos grupos** se puede utilizar una **prueba t de Student**.
- El objetivo es determinar si existe una **diferencia significativa** entre las medias de los grupos.

Hipótesis nula (H_0):

$$H_0 : \mu_{\text{Kama}} = \mu_{\text{Rosa}}$$

- No hay diferencia significativa entre las medias del área entre los grupos “Kama” y “Rosa”.

Hipótesis alternativa (H_1):

$$H_1 : \mu_{\text{Kama}} \neq \mu_{\text{Rosa}}$$

- Existe una diferencia significativa entre las medias del área entre los grupos “Kama” y “Rosa”.

Prueba t de Student para Comparar Grupos

```

1 # Crear una lista para almacenar los resultados de la prueba t para cada par de clases
2 t_test_results <- list()
3
4 # Combinaciones de clases
5 class_combinations <- combn(levels(seeds$class), 2)
6
7 # Iterar sobre cada par de clases
8 for (i in seq_len(ncol(class_combinations))) {
9   # Extraer las dos clases de la combinación
10  class_pair <- class_combinations[, i]
11  # Filtrar el data frame para las clases actuales
12  seeds_sub <- subset(seeds, class %in% class_pair)
13  # Realizar la prueba t para las clases actuales
14  t_test_result <- t.test(area ~ class, data = seeds_sub)
15  # Almacenar el resultado en la lista
16  t_test_results[[paste(class_pair, collapse = "-")] <- t_test_result
17 }
18 # Imprimir los resultados
19 t_test_results

```

\$`Kama-Rosa`

Welch Two Sample t-test


```
data: area by class
t = -17.761, df = 134.24, p-value < 2.2e-16
alternative hypothesis: true difference in means between group Kama and group Rosa is not equal
to 0
95 percent confidence interval:
 -4.445258 -3.554456
sample estimates:
mean in group Kama mean in group Rosa
      14.33443      18.33429
```

Resultados de la Prueba t - Comparación de Grupos

En una **prueba t de Student**, se establece un nivel de significancia ($\alpha = 0.05$). Si el valor p es menor que este umbral, los resultados son **estadísticamente significativos**.

Valor p bajo (cercano a 0): Esto indica una fuerte evidencia para **rechazar la hipótesis nula** y concluir que hay una diferencia significativa entre las medias de los grupos “Kama” y “Rosa”.

Por lo tanto, los datos sugieren que las muestras de semillas de las variedades **“Kama” y “Rosa” no provienen de la misma población**.

Interpretación:

Si las semillas de ambas variedades provinieran de una población homogénea, cualquier diferencia en características como el área o la longitud sería atribuible al azar.

Sin embargo, las **diferencias significativas** encontradas sugieren que las semillas de “Kama” y “Rosa” tienen **características distintivas**, lo que indica que pertenecen a **poblaciones distintas**.

Test de Normalidad (Shapiro-Wilk)

El **Test de Shapiro-Wilk** se utiliza para evaluar si un conjunto de datos sigue una **distribución normal**.

Hipótesis del Test:

Hipótesis nula (H_0): Los datos siguen una distribución normal.

Hipótesis alternativa (H_1): Los datos no siguen una distribución normal.

Interpretación del p-value:

Si el **p-value** es menor que el nivel de significancia (por ejemplo, **0.05**):

Se **rechaza H_0** , lo que indica que **los datos no siguen una distribución normal**.

Si el **p-value** es mayor que el nivel de significancia:

No hay suficiente evidencia para rechazar H_0 , lo que sugiere que **los datos pueden seguir una distribución normal**.

Conclusión:

El valor p nos indica si los datos se desvían significativamente de la normalidad.

Este test es comúnmente utilizado como paso preliminar antes de aplicar técnicas estadísticas que asumen normalidad en los datos.

Test de Normalidad

```
1 # Realizar el test de normalidad de Shapiro-Wilk para cada variable numérica en el dataframe
2 shapiro_test_results <- lapply(numeric_seeds, shapiro.test)
3
4 shapiro_test_results
```

\$area

Shapiro-Wilk normality test

data: X[[i]]

W = 0.93259, p-value = 2.948e-08

\$perimeter

Shapiro-Wilk normality test

data: X[[i]]

W = 0.93616, p-value = 5.902e-08

Test de Normalidad

```
1 # Resultados
2 for (i in seq_along(shapiro_test_results)) {
3   variable_name <- names(numeric_seeds)[i]
4   test_result <- shapiro_test_results[[i]]
5   cat("Variable:", variable_name, "\n")
6   cat("Test Statistic:", test_result$statistic, "\n")
7   cat("p-value:", test_result$p.value, "\n")
8   # Se usa la función ifelse() para determinar si p < al umbral de significancia
9   cat("Normality Test:", ifelse(test_result$p.value < 0.05, "Not Normal", "Normal"), "\n")
10  cat("\n")
11 }
```

Variable: area
Test Statistic: 0.9325935
p-value: 2.947712e-08
Normality Test: Not Normal

Variable: perimeter
Test Statistic: 0.9361616
p-value: 5.90217e-08
Normality Test: Not Normal

Variable: compactness
Test Statistic: 0.9730421
p-value: 0.0004696481
Normality Test: Not Normal

La prueba F es útil para probar si dos varianzas poblacionales son iguales

Hipótesis Nula (H_0): Las varianzas de las áreas en las clases “Kama” y “Rosa” son iguales.

Hipótesis Alternativa (H_1): Las varianzas de las áreas en las clases “Kama” y “Rosa” son diferentes.

```

1 # Lista para almacenar los resultados de las pruebas de varianza
2 var_test_results <- list()
3
4 # Iterar sobre cada par de clases
5 for (i in seq_len(ncol(class_combinations))) {
6   # Extraer las dos clases de la combinación
7   class_pair <- class_combinations[, i]
8
9   # Filtrar el dataframe para las clases actuales
10  seeds_subset <- subset(seeds, class %in% class_pair)
11
12  # Realizar la prueba de varianza
13  var_test_result <- var.test(area ~ class, data = seeds_subset)
14
15  # Almacenar el resultado en la lista
16  var_test_results[[paste(class_pair, collapse = "-")] <- var_test_result
17 }
18
19 # Imprimir los resultados

```

Resultados para la combinación: Kama-Rosa

F test to compare two variances

data: area by class

F = 0.71324, num df = 69, denom df = 69, p-value = 0.163

alternative hypothesis: true ratio of variances is not equal to 1

95 percent confidence interval:

0.4431861 1.1478428

sample estimates:

ratio of variances

0.7132377

Resultados para la combinación: Kama-Canadian

.....

Técnicas básicas

Regresión lineal

```
1 library(caret)
2 # Paso 1: Seleccionar variables predictoras y la variable de respuesta
3 variables_predictoras <- c("perimeter", "length", "width", "AC", "lengthGroove") # Variables
4 variable_respuesta <- "area" # Variable de respuesta
5
6 # Paso 2: Dividir los datos en conjuntos de entrenamiento y prueba
7 set.seed(123) # Para reproducibilidad
8 indices_entrenamiento <- createDataPartition(seeds$area, p = 0.8, list = FALSE)
9 datos_entrenamiento <- seeds[indices_entrenamiento, ]
10 datos_prueba <- seeds[-indices_entrenamiento, ]
11
12 # Paso 3: Entrenar el modelo de regresión lineal
13 modelo <- lm(formula = paste(variable_respuesta, "~", paste(variables_predictoras, collapse =
14
15 # Mostrar resumen del modelo
16 summary(modelo)
```

Call:

```
lm(formula = paste(variable_respuesta, "~", paste(variables_predictoras,
  collapse = " + ")), data = datos_entrenamiento)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.30981	-0.07706	-0.00793	0.07234	0.41636

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	-15.552375	0.161763	-96.143	< 2e-16	***
perimeter	1.529360	0.080834	18.920	< 2e-16	***
length	-0.170134	0.157289	-1.082	0.280989	
width	2.375521	0.128692	18.459	< 2e-16	***



```
lm(formula = area ~ perimeter + width + lengthGroove, data = seeds)
```

Min	1Q	Median	3Q	Max
-0.38175	-0.08258	-0.00136	0.07768	0.34984

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	-15.72546	0.11446	-137.394	< 2e-16	***
perimeter	1.45392	0.03915	37.139	< 2e-16	***
width	2.50441	0.09285	26.971	< 2e-16	***
lengthGroove	0.23005	0.05144	4.472	1.28e-05	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1					

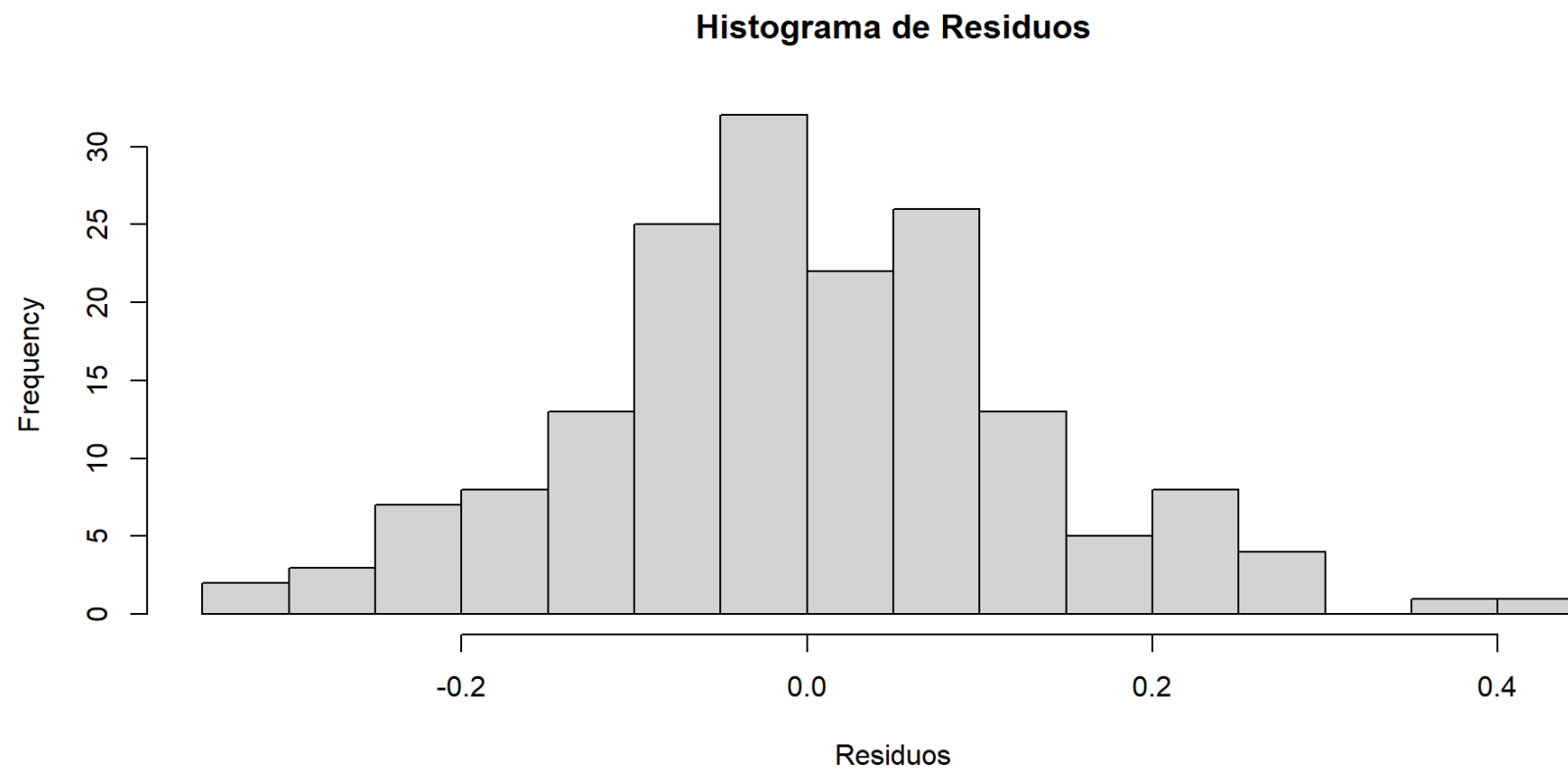
```

1 # Predecir los valores de área utilizando el modelo con los datos de prueba
2 predicciones <- predict(modelo_significativo, newdata = datos_prueba)
3
4 # Comparar las predicciones con los valores reales en los datos de prueba
5 resultados <- data.frame(Real = datos_prueba$area, Prediccion = predicciones)
6 print(resultados)

```

	Real	Prediccion
3	14.29	14.22742
6	14.38	14.36942
9	16.63	16.78186
16	14.59	14.48353
19	14.70	14.68447
21	14.16	14.23797
24	12.08	12.00407
25	15.01	15.01163
29	14.11	14.11676
40	14.28	14.28655
48	14.99	15.09145
54	14.33	14.24985
59	15.38	15.51277
60	12.11	12.49175
66	12.00	12.77350

```
1 # Ajustar el modelo de regresión lineal
2 modelo <- lm(area ~ perimeter + length + width + AC + lengthGroove, data = datos_entrenamiento)
3
4 # Obtener los residuos del modelo
5 residuos <- resid(modelo)
6
7 # Crear un histograma de los residuos
8 hist(residuos, breaks = 20, main = "Histograma de Residuos", xlab = "Residuos")
```



Matriz de Confusión para Evaluar Modelos

- Una **matriz de confusión** es una herramienta útil para evaluar el rendimiento de un modelo de clasificación, mostrando el número de **predicciones correctas** y **errores** en las diferentes categorías.

Componentes de la Matriz de Confusión:

- **Verdaderos Positivos (TP):** Predicciones correctas de la clase positiva.
- **Falsos Positivos (FP):** Predicciones incorrectas donde se predijo la clase positiva pero es negativa.
- **Falsos Negativos (FN):** Predicciones incorrectas donde se predijo la clase negativa pero es positiva.
- **Verdaderos Negativos (TN):** Predicciones correctas de la clase negativa.

```

1 # Librería para dividir datos y evaluación
2 library(caret)
3
4 # Entrenar un modelo de clasificación (ej. k-Nearest Neighbors para ilustración)
5 modelo_knn <- train(class ~ perimeter + length + width + AC + lengthGroove, data = datos_entrenamiento)
6
7 # Hacer predicciones en los datos de prueba
8 predicciones <- predict(modelo_knn, newdata = datos_prueba)
9
10 # Crear la matriz de confusión
11 confusionMatrix(predicciones, datos_prueba$class)

```

Confusion Matrix and Statistics

	Reference		
Prediction	Kama	Rosa	Canadian
Kama	13	1	3
Rosa	1	12	0
Canadian	1	0	9

Overall Statistics

```

Accuracy : 0.85
 95% CI : (0.7016, 0.9429)
No Information Rate : 0.375
P-Value [Acc > NIR] : 8.368e-10

```

